

FICHEROS EN PYTHON

Persistencia de datos en ficheros en Python

Introducción

En este tema se explicará el tratamiento de los ficheros de datos y su persistencia en disco. Se verán diferentes tipos de ficheros de datos que se utilizan y los diferentes formatos disponibles de las librerías del núcleo de Python.

Un fichero se utiliza para guardar la información e importarla a memoria cuando el programa se inicialice a medida que la vaya necesitando, aprovechando en todo momento los recursos de la máquina.

Ficheros de texto plano

Es el primer tipo de ficheros que se va a explicar son los que guardan la información en texto plano. Esto significa que con cualquier editor se podrá ver y ver si contenido. En este caso se trata de una secuencia de caracteres en una codificación específica y con ciertas características comunes pero expuestas al público.

Pueden existir de cuatro tipos:

Ficheros sin estructura básica: Son ficheros de texto que no presentan ninguna estructura interna, son sólo texto guardado en un fichero.

Con ancho fijo: son ficheros de texto que presentan un ancho definido para cada campo.

Ficheros tabulados: son ficheros que presentan un delimitador definido en todo el contenido, tanto para separar líneas como para separar recursos dentro de las líneas. Líneas de caracteres separadas por `;` o el formato CSV en el que cada línea está separada por saltos de líneas y cada columna se define por un delimitador común que en este caso es el `;`.

Ficheros en un formato estándar: con el propósito de poder representar información de forma coordinada entre quien escribe la información y quien la quiere leer.

Trabajar con fichero sin estructura

Cuando se trabaja con ficheros la primera opción a realizar es abrirlo para poder trabajar con el. La función que permite abrir un fichero es `open()`. Esta función devuelve un objeto archivo, mediante el cual podemos interactuar con el archivo. Esta función se utiliza principalmente con dos parámetros de la siguiente forma:

`open(file, modo)`

file: el único obligatorio, es una cadena de texto que contiene el nombre del archivo (incluida la ruta si no se encuentra en el directorio actual)

modo: opcional, es una cadena que indica el modo en el que el archivo debe ser abierto y es una combinación de los siguientes:

`r`: modo lectura. Es el modo por defecto, y cuando no se combina con ningún otro es equivalente a la lectura de texto plano o `rt`.

`w`: escritura. Permite escribir en un fichero, pero trunca el contenido que había anteriormente en el mismo. Si se abre con

esta opción y el fichero no existe se crea, pero si ya existía se sobrescribe.

'x': modo de apertura del fichero exclusivamente para nueva creación. Si el fichero ya existe, elevará una excepción.

'a': modo de escritura añadiendo contenido al final del fichero.

'b': apertura del fichero en modo binario. Tanto para lectura **'rb'** como escritura **'wb'**. Devuelve el contenido en objetos de tipo **bytes**.

't': apertura del fichero en modo texto. Tanto para la lectura **'r'** o **'rt'** como escritura **'w'** o **'wt'** de texto plano. Devuelve el contenido en objetos de tipo **str**.

'+': apertura del fichero para actualización, tanto para lectura como escritura,

Ejemplo de como abrir un fichero de datos:

```
01-abrir-fichero.py > ...
1 fichero = open('ficheros/fichero01.txt')
2 print(f'El fichero ha sido abierto en modo: {fichero.mode}')
3 fichero.close()
```

De esta forma se abre un fichero, si en la llamada no se especifica el segundo parámetro por defecto se abre en modo lectura. El resultado de la ejecución del código anterior es el siguiente.

```
Py
El fichero ha sido abierto en modo: r
• rgh.informatica@macbpro-rgh Unidad06 %
```

La tercera línea del código cierra el flujo de datos que representa el fichero. Hay que tener en cuenta que los ficheros deben cerrarse antes de terminar la ejecución del programa. Si el fichero se abre en modo escritura, esto garantiza el vaciado al disco de lo que esté almacenando el buffer, consolidándose así la información en el fichero.

A tener en cuenta:

En la función `open()` si en el primer parámetro sólo se pone el nombre del fichero, éste deberá estar en el mismo directorio donde se encuentra el fichero que se está ejecutando.

Si se utiliza cualquier editor de código como Visual Studio, Atom o similares, se debe verificar en la terminal cual es la ruta por defecto que tiene, ya que al ejecutar el fichero Python es ahí donde se va a buscar el fichero cuyo nombre se haya puesto en la función `open()`.

Una forma de que no aparezca error es utilizar la ruta completa hasta el fichero, pero si el código se debe ejecutar en otra máquina probablemente puede dar error si la ruta no coincide.

Otra buena práctica para realizar lo mismo es el uso del comando `with`, (forma pitonisa de trabajar con ficheros) un encapsulador de conceptos a la hora de abrir el fichero, esto garantiza que el flujo de datos se cierre evitando el uso de la función `close` para tener que cerrarlo.

```
02-abrir-fichero-with.py > ...  
1  # uso de with para el trabajo con ficheros  
2  # al salir de la sentencia with el fichero es cerrado  
3  
4  with open('ficheros/fichero01.txt', 'r') as fichero:  
5      print(f'El fichero ha sido abierto en modo: {fichero.mode}')
```

En el ejemplo anterior, se usa el segundo parámetro para abrir el fichero en modo lectura `'r'`, si queremos abrirlo para modo escritura se pueden usar las opciones: `'w'`, `'x'`, y `'a'` explicadas anteriormente con las características propias de cada una.

Para evitar que el programa termine abruptamente por problemas con la apertura de ficheros se utiliza El método try de la siguientes dos formas:

```
03-abrir-fichero-uso-try.py > ...
1  try:
2      fichero = open('ficheros/fichero01.txt', 'r')
3  except:
4      print("Ha ocurrido un error con el fichero\n")
5  else:
6      # aquí se pueden realizar las operaciones con el fichero
7      print(f'El fichero ha sido abierto en modo: {fichero.mode}')
8      fichero.close()
```

Si utilizamos la cláusula with entonces el código queda

```
04-abrir-fichero-with-try.py > ...
1  try:
2      # aquí se abre y realizan las operaciones con el fichero
3      with open('ficheros/fichero01.txt', 'r') as fichero:
4          print(f'El fichero ha sido abierto en modo: {fichero.mode}')
5  except:
6      print("Ha ocurrido un error con el fichero\n")
7  else:
8      # aquí no se pueden realizar operaciones con el fichero
9      # fue cerrado al salir del with
10     pass
```

Si cambiamos el nombre de fichero que queremos abrir (no se encuentra en el directorio), el resultado de la ejecución sería el siguiente:

```
Ha ocurrido un error con el fichero
rgh.informatica@macbpro-rgh Unidad06 %
```

Lectura de ficheros de texto

Las instrucciones que permiten leer datos de un fichero son: `read()`, `readline()` y `readlines()`, veamos ejemplos de cada una de ellas.

Veamos varios ejemplos:

Uso de `read()` sin parámetros

```
05-leer-fichero-read.py > ...
1  try:
2      fichero = open('ficheros/fichero01.txt', 'r')
3  except:
4      print("Ha ocurrido un error con el fichero\n")
5  else:
6      texto = fichero.read()
7      print(texto)
8      fichero.close()
```

En el ejemplo en la línea 6 se usa El método `read()` en este caso sin parámetro, por tanto el contenido completo del fichero se almacena en la variable `texto`. En la línea 7 se imprime dicha variable y posteriormente se cierra el fichero .

El resultado de la ejecución del código es el siguiente:

```
esta es la línea 01 del fichero
creamos comentarios
esta es la línea final.

otra vez escribo

rgh.informatica@macbpro-rgh Unidad06 %
```

Uso de read() con parámetro

En esta caso la llamada tendrá la forma `read(size)` donde `size` es el número de bytes a leer.

Veamos el siguiente ejemplo:

```
05-leer-fichero-read-nbytes.py > ...
1  # muestra contenido de un fichero por pantalla
2  try:
3      fichero = open('ficheros/fichero02.txt', 'r')
4  except:
5      print("Ha ocurrido un error con el fichero\n")
6  else:
7      caracter = fichero.read(1)
8      while caracter != "":
9          print(caracter)
10         caracter = fichero.read(1)
11     fichero.close()
```

El contenido del fichero `fichero02.txt` es el siguiente:

```
ficheros > ≡ fichero02.txt
1  hola
2  cómo estas?
```

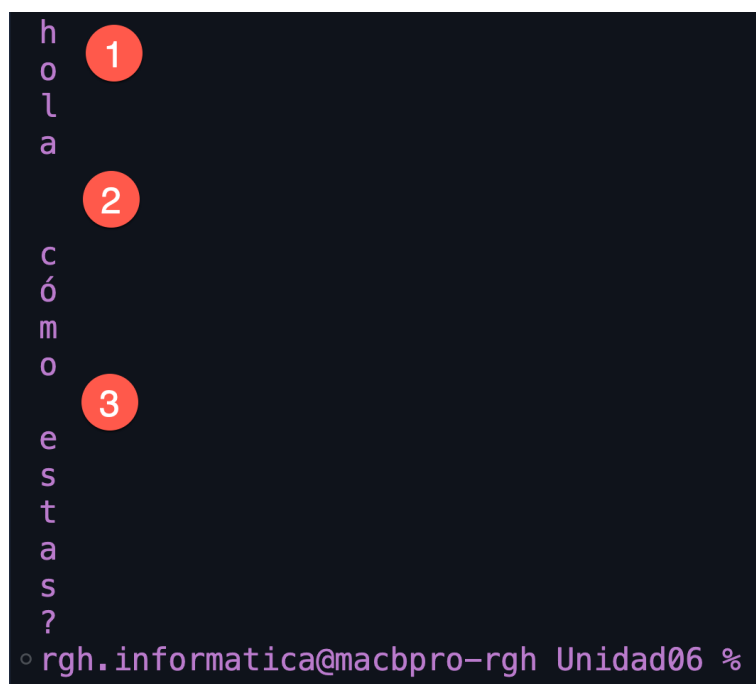
El código del ejemplo anterior, abre el fichero para su lectura y ejecuta El método `read(1)` para leer 1 sólo carácter a la vez.

En la línea 8 se utiliza un bucle `while` con la condición de que el carácter leído sea distinto de `""`, esto se realiza porque cuando el

fichero se acabe y no haya más caracteres que leer El método devolverá el string "".

Dentro del bucle se imprime el carácter leído y se procede a leer el siguiente, esto se realizará repetidamente hasta que se alcance el final del fichero.

La salida mostrada al finalizar la ejecución del código anterior es la siguiente:



```
h
o
l
a
c
ó
m
o
e
s
t
a
s
?
rgh.informatica@macbpro-rgh Unidad06 %
```

1. A la hora de imprimir `print(carácter)` hace un salto de línea por eso se muestra un carácter por fila
2. Al finalizar la primera línea del fichero también se lee el carácter de fin de línea (`'\n'`), es por eso que al escribirlo se produce un salto de dos líneas.
3. El carácter `' '` (espacio en blanco) como cualquier carácter también se lee y al imprimirse se hace el salto.

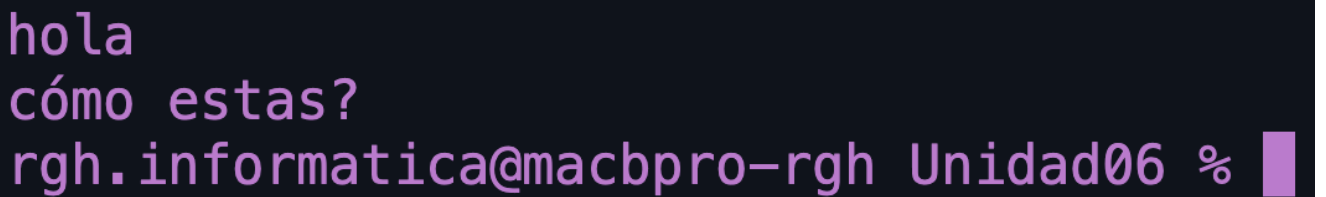
Vamos a mejorar el código anterior para que al mostrarlo por pantalla se vea igual que el original, el resultado es el siguiente código:

 05-leer-fichero-read-nbytes2.py > ...

```
1  try:
2      fichero = open('ficheros/fichero02.txt', 'r')
3  except:
4      print("Ha ocurrido un error con el fichero\n")
5  else:
6      caracter = fichero.read(1)
7      while caracter != "":
8          print(caracter, end='')
9          caracter = fichero.read(1)
10     print()
11     fichero.close()
```

Se ha hecho un cambio en la línea 8, para que El método `print()` no realice el salto de línea. Se agrega la línea 10, para que al terminar se salte de línea y el prompt de la terminal se muestre de forma correcta.

El resultado es el que se muestra a continuación:



```
hola
cómo estas?
rgh.informatica@macbpro-rgh Unidad06 %
```

Uso de `readline()`

Otra forma de leer datos de un fichero es hacerlo línea a línea, esto se puede realizar mediante el uso de El método `readline()`.

Veamos un ejemplo de como usarlo, para ello leeremos el "fichero02.txt" y mostrarlo por pantalla.

La forma de leer con El método `readline()` es similar a la de `read()`:

```
05-leer-fichero-readline.py ×
05-leer-fichero-readline.py > ...
1  # uso de readline para la lectura de un fichero
2  # el programa lee línea a línea el fichero y
3  # lo muestra por pantalla
4
5  try:
6      fichero = open('ficheros/fichero02.txt', 'r')
7      linea = fichero.readline()
8      while linea != "":
9          print(linea)
10         linea = fichero.readline()
11     fichero.close()
12 except:
13     print("Ha ocurrido un error con el fichero\n")
14 else:
15     print("Fin de ejecución...")
```

- Lee todos los caracteres de una línea del fichero y lo almacena en el string 'línea'
- Al leer también se lee el carácter de final de línea '\n'
- Al terminar la lectura del fichero se retorna la cadena vacía "".
-

La salida que se obtiene al ejecutar el código anterior muestra el mismo problema que la primera versión de `read()`, el salto de línea provocado por la lectura del carácter '\n'

```
hola
1
cómo estas?
Fin de ejecución...
rgh.informatica@macbpro-rgh Unidad06 %
```

Las mejoras en el código son similares al caso de read()

```
05-leer-fichero-readline2.py > ...
1  # uso de readline para la lectura de un fichero
2  # el programa lee línea a línea el fichero y
3  # lo muestra por pantalla
4
5  try:
6      fichero = open('ficheros/fichero02.txt', 'r')
7      linea = fichero.readline()
8      while linea != "":
9          print(linea, end='')  1
10         linea = fichero.readline()
11     fichero.close()
12 except:
13     print("Ha ocurrido un error con el fichero\n")
14 else:
15     print("\nFin de ejecución...")  2
```

(1): Se evita el salto de línea en El método print()

(2): Se añade '\n' al comienzo de la escritura para hacer el salto y se escriba en una nueva línea.

Uso de readlines()

El método readlines() a diferencia de readline() lee todo el fichero línea a línea y lo deja almacenado como una lista, en la que cada elemento representa una línea del fichero.

Veamos un ejemplo de su uso

```
05-leer-fichero-readlines.py > ...
1  # uso de readlines para la lectura de un fichero
2
3  try:
4      fichero = open('ficheros/fichero02.txt', 'r')
5      textoaux = fichero.readlines()
6      for linea in textoaux[:-1]:
7          print(linea[:-1])
8      print(textoaux[-1])
9  except:
10     print("Ha ocurrido un error con el fichero\n")
11 else:
12     print("Fin de ejecución...")
```

En la línea 5 se usa la variable `textoaux` como una lista donde se almacenarán todas las líneas leídas del fichero.

En la líneas 6-8 se recorre la lista, hay que darse cuenta que todas las líneas menos la última terminan con el carácter `'\n'`. Por tanto por un lado recorreremos con el `for` todas las líneas menos la última y luego escribimos la última.

La salida obtenida es la misma del ejemplo anterior

```
hola
cómo estas
Fin de ejecución...
rgh.informatica@macbpro-rgh Unidad06 %
```

Escritura de ficheros de texto

Hay dos instrucciones que permiten escribir texto en un fichero: `write()` y `writelines()`. Veamos a continuación ejemplos de su uso.

Uso de `write()`

Para escribir en un fichero se usa el método `write(a_escribir)`, que escribe el texto `'a_escribir'` en el fichero, sin hacer salto de línea.

Veamos un ejemplo, en el que se leen líneas del teclado y se crea un fichero con lo recibido. Para terminar se usa `<intro>`.

```
07-escribir-fichero01.py > ...
1  # se leen líneas del teclado y se escriben a un fichero
2
3  try:
4      fichero = open('ficheros/pruebaescritura.txt', 'w')
5  except:
6      print("Ha ocurrido un error con el fichero\n")
7  else:
8      while True:
9
10         # tener cuidado que al leer de teclado el \n no se almacena
11
12         texto = input('Línea de texto ->: ')
13         if texto == '':          # para terminar el bucle
14             break
15         fichero.write(texto + '\n') # hay que forzar el \n en la escritura
16     fichero.close()
```

La ejecución del programa

```
● rgh.informatica@macbpro-rgh Unidad06 % ./us
escribir-fichero01.py
Línea de texto ->: primera línea
Línea de texto ->: segunda línea
Línea de texto ->: línea final
Línea de texto ->:
○ rgh.informatica@macbpro-rgh Unidad06 %
```

El fichero generado

```
ficheros > ≡ pruebaescritura.txt
1  primera línea
2  segunda línea
3  línea final
```

Tener en cuenta que el fichero tiene 4 líneas, el intro final de la tercera línea.

Vemos otro ejemplo, ahora par copiar un fichero de texto letra a letra.

En este caso hay que tener en cuenta que no se debe escribir el '
' en cada línea porque se quiere hacer un copia del fichero original.

El código es el siguiente:

```
07-escribir-fichero02.py > ...
1  # copiar un fichero en otro caracter a caracter
2
3  try:
4      fichero_lectura = open('ficheros/fichero01.txt', 'r')
5  except:
6      print("Ha ocurrido un error con el fichero de lectura\n")
7  else:
8      try:
9          fichero_escritura = open('ficheros/fichero01.bckp', 'w')
10     except:
11         print("Ha ocurrido un error con el fichero de escritura\n")
12     else:
13         # aquí se pueden realizar las operaciones con el fichero
14
15         # modificar
16         car_leido = fichero_lectura.read(1)
17         while car_leido != "":
18             fichero_escritura.write(car_leido)
19             car_leido = fichero_lectura.read(1)
20
21         fichero_escritura.close()
22         fichero_lectura.close()
```

Los ficheros original y copiado son los siguientes:

<pre>ficheros > ≡ fichero01.bckp 1 esta es la línea 01 del fichero 2 creamos comentarios 3 esta es la línea final. 4 5 otra vez escribo</pre>	<pre>ficheros > ≡ fichero01.txt 1 esta es la línea 01 del fichero 2 creamos comentarios 3 esta es la línea final. 4 5 otra vez escribo 6</pre>
--	---

Uso de writelines()

Este método writelines(lista), recibe como parámetro una lista y escribe en el fichero cada uno de los elementos de la lista. La escritura es continuada por lo que hay que forzar los saltos de líneas que se requieran.

Veamos su uso

```
07-escribir-fichero03.py > ...
1  # usando writelines()
2
3  lista = ['línea 01', 'línea 03', 'línea 03']
4
5  try:
6      fichero_escritura = open('ficheros/fichero03.txt', 'w')
7  except:
8      print("Ha ocurrido un error con el fichero de escritura\n")
9  else:
10     # aquí se pueden realizar las operaciones con el fichero
11
12     fichero_escritura.writelines(lista)
13     fichero_escritura.close()
```

Cuya salida se muestra a continuación

```
ficheros > ≡ fichero03.txt
1  línea 01línea 03línea 03
```

Como se puede apreciar la escritura es continua. Para que cada elemento de la lista se escriba en una línea se debe incorporar a cada elemento.

Cambiando la línea 3 por lo siguiente

```
lista = ['línea 01\n', 'línea 02\n', 'línea 03']
```

Al ejecutar el programa el resultado obtenido es el siguiente:

```
ficheros > ≡ fichero03.txt
1  línea 01
2  línea 02
3  línea 03
```

Ficheros de texto CSV (usando librería)

En Python existe la librería `csv` que permite trabajar con ficheros `csv`. La documentación de esta librería se puede encontrar en: <https://docs.python.org/3/library/csv.html>

Veamos como podemos escribir y leer listas y diccionarios con esta librería.

Escritura de una lista en un fichero csv

La librería que se debe importar es `csv`.

Veamos un ejemplo del código siguiente:

```
08-csv-escribirlistas.py > ...
1  import csv
2
3  # datos en una lista para generar fichero .csv
4
5  contactos = [
6      ("Manuel", "Desarrollador Web", "manuel@ejemplo.com"),
7      ("Lorena", "Gestora de proyectos", "lorena@ejemplo.com"),
8      ("Javier", "Analista de datos", "javier@ejemplo.com"),
9      ("Marta", "Experta en Python", "marta@ejemplo.com") ]
10
11  with open ("ficheros/contactos.csv", "w", newline="\n") as csvfile:
12
13      # creamos un objeto de la clase
14      writer = csv.writer(csvfile, delimiter = ",")
15      for contacto in contactos:
16          writer.writerow(contacto)
```

En la línea 11 se fuerza a que el carácter de fin de línea sea `<intro>`, aunque por defecto ya lo es.

En la línea 14 se crea un objeto escritor que se encarga de dar formato a los datos del usuario en cadenas delimitadas a un archivo dado, para ello se pasa como parámetros el identificador del fichero abierto y el delimitador que se quiera usar, para el ejemplo se usa `','`.

Luego se recorre la lista y se escribe en el fichero mediante el método `writerow()` al que se pasa como parámetro un elemento de la lista.

El contenido del fichero resultante de la ejecución del código anterior es el siguiente:

```
• rgh.informatica@macbpro-rgl Unidad06 % cat ficheros/contactos.csv
Manuel,Desarrollador Web,manuel@ejemplo.com
Lorena,Gestora de proyectos,lorena@ejemplo.com
Javier,Analista de datos,javier@ejemplo.com
Marta,Experta en Python,marta@ejemplo.com
○ rgh.informatica@macbpro-rgl Unidad06 % █
```

Lectura de elementos de lista en un fichero csv

El código para leer un fichero `'csv'` mediante la librería es el siguiente:

```
08-csv-leerparalista.py > ...
1  import csv
2  lista = []
3
4  with open ("ficheros/contactos.csv", newline="\n") as csvfile:
5      reader = csv.reader(csvfile, delimiter = ",")
6
7      # en reader está todo lo leído cada elemento es un fil
8
9      for nombre, empleo, email in reader:
10
11          # imprimir en pantalla
12          print (nombre, empleo, email)
13
14          # generar de nuevo la lista original
15          lista.append((nombre, empleo, email))
16
17  print ('*****')
18  for elemento in lista:
19      print(elemento)
20
```

La línea 5, retorna un objeto reader que iterará sobre las líneas del fichero proporcionado. Este fichero se asoció a la variable 'csvfile'. Luego recorriendo con un for obtenemos cada una de las líneas del fichero.

La salida del código anterior es:

```
• rgh.informatica@macbpro-rgh Unidad06 % /usr/local/bin/pyt
-leerparalista.py
Manuel Desarrollador Web manuel@ejemplo.com
Lorena Gestora de proyectos lorena@ejemplo.com
Javier Analista de datos javier@ejemplo.com
Marta Experta en Python marta@ejemplo.com
*****
('Manuel', 'Desarrollador Web', 'manuel@ejemplo.com')
('Lorena', 'Gestora de proyectos', 'lorena@ejemplo.com')
('Javier', 'Analista de datos', 'javier@ejemplo.com')
('Marta', 'Experta en Python', 'marta@ejemplo.com')
• rgh.informatica@macbpro-rgh Unidad06 %
```

Donde se muestra la salida por pantalla, tanto del texto leído como de los elementos de la lista originada como la original del ejercicio anterior.

Escritura de un diccionario en un fichero csv

En este caso se debe crear una lista con los campos que va a tener el diccionario, el método a usar ahora es `DictWriter()` al que se le debe pasar dos parámetros, el flujo del fichero y la lista de campos (línea 9) de los datos que se van a escribir.

```
1  import csv
2
3  contactos = [
4      ("Manuel", "Desarrollador Web", "manuel@ejemplo.com"),
5      ("Lorena", "Gestora de proyectos", "lorena@ejemplo.com"),
6      ("Javier", "Analista de datos", "javier@ejemplo.com"),
7      ("Marta", "Experta en Python", "marta@ejemplo.com") ]
8
9  campos = ["nombre", "empleo", "email"]
10 with open("ficheros/contactosdict.csv", "w", newline = "\n") as csvfile:
11
12     # se necesita esta lista para pasarla como parámetro en fieldnames
13     writer = csv.DictWriter(csvfile, fieldnames = campos)
14
15     # escribe un encabezado con los elementos de la lista campos
16     writer.writeheader()
17
18     for nombre, empleo, email in contactos:
19         # en cada iteración se genera un diccionario que se escribe en el fichero
20         writer.writerow({ "nombre": nombre, "empleo": empleo, "email": email })
```

En la línea 16, se utiliza el método `writeheader()` para escribir una cabecera en el fichero.

Para escribir una fila se utiliza `writerow()`. En la línea 20 se crea el diccionario antes de escribirlo al fichero,

El contenido del fichero generado con la ejecución del código anterior es el siguiente:

```
• rgh.informatica@macbpro-rgh Unidad06 % cat ficheros/contactosdict.csv
nombre,empleo,email
Manuel,Desarrollador Web,manuel@ejemplo.com
Lorena,Gestora de proyectos,lorena@ejemplo.com
Javier,Analista de datos,javier@ejemplo.com
Marta,Experta en Python,marta@ejemplo.com
○ rgh.informatica@macbpro-rgh Unidad06 %
```

La primera línea de texto es lo que se considera el encabezado y sirve como información para cuando se quiera leer este fichero de datos, del número de campos que debe leer en las siguientes filas.

Lectura de un fichero csv con cabecera

En este caso el fichero .csv tiene un encabezado que representa los nombres de los campos que intervienen en los datos.

```
• rgh.informatica@macbpro-rgh Unidad06 % cat ficheros/contactosdict.csv
nombre,empleo,email
Manuel,Desarrollador Web,manuel@ejemplo.com
Lorena,Gestora de proyectos,lorena@ejemplo.com
Javier,Analista de datos,javier@ejemplo.com
Marta,Experta en Python,marta@ejemplo.com
○ rgh.informatica@macbpro-rgh Unidad06 %
```

Para procesarlo se usa un código como el que se muestra a continuación:

```
08-csv-leerdiccionario.py > ...
1  import csv
2
3  with open("ficheros/contactosdict.csv", "r", newline = "\n") as csvfile:
4
5      reader = csv.DictReader(csvfile, delimiter=",")
6      for fila in reader:
7          # en cada iteración se escribe un diccionario que se muestra por pantalla
8          print(fila)
9          print(f'\tnombre: {fila["nombre"]}\n\templeo: {fila["email"]} \n\tcorreo:{fila["email"]}')

```

Se usa el método `DictReader()`, al que se le pasa como parámetro el flujo de datos y el delimitador que tienen separa los campos.

Destacar que automáticamente se lee la línea de encabezado y que cada línea leída del fichero es transformada en un diccionario tal y como lo muestra la impresión realizada en la línea 8.

Con la línea 9 se realiza una impresión más elegante de los datos del diccionario.

Cabe destacar que esos diccionarios generados se pueden almacenar en una lista.

El resultado de la ejecución del código anterior es:

```
{'nombre': 'Manuel', 'empleo': 'Desarrollador Web', 'email': 'manuel@ejemplo.com'}  
    nombre: Manuel  
    empleo: manuel@ejemplo.com  
    correo:manuel@ejemplo.com  
{'nombre': 'Lorena', 'empleo': 'Gestora de proyectos', 'email': 'lorena@ejemplo.com'}  
    nombre: Lorena  
    empleo: lorena@ejemplo.com  
    correo:lorena@ejemplo.com  
{'nombre': 'Javier', 'empleo': 'Analista de datos', 'email': 'javier@ejemplo.com'}  
    nombre: Javier  
    empleo: javier@ejemplo.com  
    correo:javier@ejemplo.com  
{'nombre': 'Marta', 'empleo': 'Experta en Python', 'email': 'marta@ejemplo.com'}  
    nombre: Marta  
    empleo: marta@ejemplo.com  
    correo:marta@ejemplo.com  
rgh.informatica@macbpro-rgh Unidad06 %
```

Ficheros de texto JSON (usando librería)

JSON que son las siglas de JavaScript Object Notation, es un formato muy utilizado para la transmisión de datos entre ordenadores.

Estructura y formato JSON

```
{
  "size": "medium",
  "price": 15.67,
  "toppings": ["mushrooms", "pepperoni", "basil"],
  "extra_cheese": false,
  "delivery": true,
  "client": {
    "name": "Jane Doe",
    "phone": null,
    "email": "janedoe@email.com"
  }
}
```

En Python existe la librería `json` que permite trabajar con ficheros con texto en este formato. La documentación de esta librería se puede encontrar en: <https://docs.python.org/3/library/json.html>

Escritura de fichero con formato JSON

Veamos como podemos escribir un fichero con formato JSON con esta librería.

```
09-json-escribir.py > ...
1  import json
2
3  contactos = [
4      ("Manuel", "Desarrollador Web", "manuel@ejemplo.com") ,
5      ("Lorena", "Gestora de proyectos", "lorena@ejemplo.com"),
6      ("Javier", "Analista de datos", "javier@ejemplo.com"),
7      ("Marta", "'Experta en Python", "marta@ejemplo.com") ]
8
9  datos = []
10
11  # creamos un diccionario con los datos de la lista
12
13  for nombre, empleo, email in contactos:
14      datos.append({"nombre":nombre, "empleo":empleo, "email":email})
15
16  with open ("ficheros/contactos.json", "w") as jsonfile:
17      json.dump(datos, jsonfile)
```

En la línea 16 abrimos el fichero para escritura, en la 17 se usa el método `dump()` para el volcado de la lista de elementos de tipo diccionario que se confeccionó antes.

`Dump()` vuelca la información del diccionario al fichero con formato JSON.

El fichero resultante de la ejecución del código anterior:

```
rg.h.informatica@macbpro-rg.h Unidad06 % cat ficheros/contactos.json
[{" nombre": "Manuel", "empleo": "Desarrollador Web", "email": "manuel@ejemplo.com"}, {" nombre": "Lorena", "empleo": "Gestora de proyectos", "email": "lorena@ejemplo.com"}, {" nombre": "Javier", "empleo": "Analista de datos", "email": "javier@ejemplo.com"}, {" nombre": "Marta", "empleo": "'Experta en Python", "email": "marta@ejemplo.com"}]
```

Que está escrito en una forma compacta, se puede expresar en un formato menos comprimido, utilizando distintos parámetros a la hora de ejecutar el método `dump()`, como por ejemplo:

```
json.dump(datos, jsonfile, sort_keys = True, indent = 4)
```

```
16 with open ("ficheros/contactos.json", "w") as jsonfile:
17     json.dump(datos, jsonfile, sort_keys = True, indent = 4)
```

El resultado de la ejecución del código anterior sería:

```
[
  {
    " nombre": "Manuel",
    "email": "manuel@ejemplo.com",
    "empleo": "Desarrollador Web"
  },
  {
    " nombre": "Lorena",
    "email": "lorena@ejemplo.com",
    "empleo": "Gestora de proyectos"
  },
  {
    " nombre": "Javier",
    "email": "javier@ejemplo.com",
    "empleo": "Analista de datos"
  },
  {
    " nombre": "Marta",
    "email": "marta@ejemplo.com",
    "empleo": "'Experta en Python"
  }
]%
```

rg.h.informatica@macbpro-rg.h Unidad06 %

Lectura de fichero con formato JSON

Un ejemplo de ejecución:

```
09-json-leer.py > ...
1  import json
2
3  with open ("ficheros/contactos.json", "r") as jsonfile:
4      datos = json.load(jsonfile)
5      for fila in datos:
6          print(fila)
7          print(f'\tnombre: {fila["nombre"]}\n\templeo: {fila["email"]} \n\tcorreo:{fila["email"]}')]
```

Se puede apreciar en la línea 4 que el método para la lectura del fichero es load() al cual se le debe pasar como parámetro el flujo de un fichero ya abierto para lectura.

El resultado mostrado por pantalla es el siguiente:

```
{'email': 'manuel@ejemplo.com', 'empleo': 'Desarrollador Web', 'nombre': 'Manuel'}
nombre: Manuel
empleo: manuel@ejemplo.com
correo:manuel@ejemplo.com
{'email': 'lorena@ejemplo.com', 'empleo': 'Gestora de proyectos', 'nombre': 'Lorena'}
nombre: Lorena
empleo: loren@ejemplo.com
correo:lorena@ejemplo.com
{'email': 'javier@ejemplo.com', 'empleo': 'Analista de datos', 'nombre': 'Javier'}
nombre: Javier
empleo: javier@ejemplo.com
correo:javier@ejemplo.com
{'email': 'marta@ejemplo.com', 'empleo': '"Experta en Python"', 'nombre': 'Marta'}
nombre: Marta
empleo: marta@ejemplo.com
correo:marta@ejemplo.com
rgh.informatica@macbpro-rgh Unidad06 %
```

FICHEROS BINARIOS

Cuando se pretende guardar contenido que no es estrictamente texto, con un formato particular, el cual solo es conocido por las aplicaciones que usan ese formato, o un formato que pueda prescindir de los saltos de línea, se opta por usar ficheros en formato binario. La principal diferencia es que el contenido de los ficheros en binario no es fácilmente reconocible y, en multitud de ocasiones, contiene datos mucho más complejos que simple texto, como pueden ser sonidos, imágenes o vídeos, entre otros muchos tipos de datos.

Los datos en los ficheros binarios se agrupan en bytes contiguos y se suelen leer como un stream de datos, ya que en la mayoría de los casos no tienen saltos de línea o separadores de bloques de contenido.

En Python no existe una librería específica para leer datos en binario, pero sí que suele haber soporte para cada formato; no importa si es un archivo de audio, video, PDF o cualquier otro formato, se deberá encontrar una librería (normalmente de terceros) que soporte ese formato específico.

Lectura y escritura de ficheros con formato binario

En este apartado veremos como realizar una copia de un fichero pero trabajando con el modo binario. El fichero a copiar es de una imagen, por tanto no se pueden tratar como si fuera un fichero de texto. Como ya se dijo al principio del tema para abrir un fichero en forma binaria se debe añadir al parámetro modo una 'b'. Por tanto si se quiere abrir un fichero para lectura se hará 'rb' y 'wb' o 'ab' si se quiere abrir para escritura.

Veamos el código que permite la copia de un fichero de imagen.

```
14-escribir-fichero04 binario.py > ...
1  # copiar un fichero en otro en modo binario
2
3  try:
4      fichero_lectura = open('ficheros/jack_russell.png', 'rb')
5  except:
6      print("Ha ocurrido un error con el fichero de lectura\n")
7  else:
8      try:
9          fichero_escritura = open('ficheros/perro.png', 'wb')
10     except:
11         print("Ha ocurrido un error con el fichero de escritura\n")
12     else:
13         byte_leido = fichero_lectura.read(1)
14         while byte_leido != b"":
15             fichero_escritura.write(byte_leido)
16             byte_leido = fichero_lectura.read(1)
17
18         fichero_escritura.close()
19         fichero_lectura.close()
20
```

En la línea 4 se abre el fichero para su lectura binaria.

En la línea 9 se abre fichero para escritura binaria

En las líneas 13 y 17 se lee del fichero 1 byte

*En la línea 14 en la condición de parada se agrega **b** antes de "" para que la cadena vacía sea interpretada en binario.*

En la línea 15 se realiza la escritura en binario.

En las líneas 18 y 19 se cierran los ficheros

El fichero resultante de la ejecución de este programa es el que se muestra a la derecha. Como se puede apreciar se ha realizado la copia de un fichero de imagen. Se puede realizar el mismo procedimiento para un fichero de audio, etc.



Veamos ahora un par de ejemplos para trabajar

Primero crearemos un fichero binario cuyo contenido será texto, este servirá como entrada para el segundo ejemplo.

El código para crear el fichero string.bin es el que se muestra a continuación

```
11-creando-binario-string.py > ...
1  # Creamos un fichero binario con contenido de caracteres
2
3  fichero = open("ficheros/string.bin", "wb")
4
5  # Escribir dos líneas de texto en modo binario
6  fichero.write(b"Welcome to LinuxHint.\nLearn Python Programming.")
7
8  fichero.close()
```

En la línea 3 se abre el fichero para escribir de forma binaria.

*En la línea 6 se escribe de forma binaria el string de dos líneas. Tener en cuenta la **b** antes del string, esto fuerza a que sea escrito de forma binaria.*

El contenido del fichero creado es el que se muestra a continuación

```
● rgh.informatica@macbpro-rgh Unidad06 % cat ficheros/string.bin
Welcome to LinuxHint.
Learn Python Programming.%
○ rgh.informatica@macbpro-rgh Unidad06 %
```

El código del programa que lee el fichero string.bin y lo muestra por pantalla leyendo 7 caracteres a la vez es el siguiente

```
11-leer-texto-en-binario.py > ...
1  # Mostramos el contenido del fichero string.bin por pantalla
2
3  fichero = open("ficheros/string.bin", "rb")
4
5  data_byte = fichero.read(7)
6
7  while data_byte:
8      print(data_byte)
9      data_byte = fichero.read(7)
```

En la línea 7, la condición del bucle es la variable que guarda los datos leídos del fichero, se saldrá del bucle cuando en una lectura no se haya leído nada.

La salida del programa es la siguiente:

```
b'Welcome'
b' to Lin'
b'uxHint.'
b'\nLearn '
b'Python '
b'Program'
b'ming.'
rgh.informatica@macbpro-rgh Unidad06 %
```

En la imagen anterior se puede apreciar que cada línea comienza con **b**, esto indica que el contenido ha sido interpretado de forma binaria, si queremos que cada lectura realizada al momento de escribirla sea interpretada como un texto se debe decodificar, para ello, modificamos la línea 8 del código dejándola así:

```
print(data_byte.decode())
```

Como se aprecia a la variable se le ejecuta el método `decode()` que transforma el binario en un string de caracteres. Ahora si volvemos a ejecutar el programa se muestra lo siguiente:

```
Welcome  
to Lin  
uxHint.  
  
Learn  
Python  
Program  
ming.  
○ rgh.informatica@macbpro-rgh Unidad06 %
```

Si ahora queremos lo que se vea en pantalla sea igual al contenido del fichero bastará con leer 1 carácter a la vez. Haciendo el cambio el programa queda de la siguiente forma:

```
11-leer-texto-en-binario-parte2.py > ...  
1  # Mostramos el contenido del fichero string.bin por pantalla  
2  
3  fichero = open("ficheros/string.bin", "rb")  
4  
5  # interpretado como no binario  
6  
7  data_byte = fichero.read(1)  
8  
9  while data_byte:  
10     print(data_byte.decode(), end = "")  
11     data_byte = fichero.read(1)
```

Y al ejecutarlo se muestra por pantalla:

```
Welcome to LinuxHint.  
Learn Python Programming.%  
rgh.informatica@macbpro-rgh Unidad06 %
```

Si probamos ahora ejecutar el comando `read()` sin parámetros

```
11-leer-texto-en-binario-parte3.py > ...  
1  # Mostramos el contenido del fichero string.bin por pantalla  
2  
3  fichero = open("ficheros/string.bin", "rb")  
4  
5  # interpretado como binario y leyendo el fichero entero  
6  
7  with open('ficheros/string.bin', 'rb') as fichero:  
8      content = fichero.read()  
9  
10 print("Imprimimos el contenido del fichero leído\n")  
11 print(content)
```

Muestra el siguiente resultado

```
Imprimimos el contenido del fichero leído  
b'Welcome to LinuxHint.\nLearn Python Programming.'  
rgh.informatica@macbpro-rgh Unidad06 %
```

Que al igual que antes se muestra en binario, para que sea interpretado como texto, antes de imprimirlo se debe decodificar, por tanto debemos cambiar la línea 11 por el siguiente texto:

```
print(content.decode())
```

Una vez realizado el cambio, volvemos a ejecutar el programa, que ahora muestra la siguiente salida:

```
Imprimimos el contenido del fichero leído

Welcome to LinuxHint.
Learn Python Programming.
rgh.informatica@macbpro-rgh Unidad06 %
```

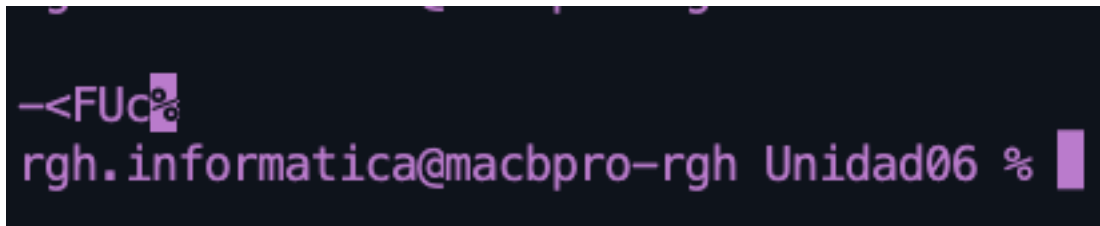
Para preparar el fichero de entrada para el segundo ejemplo escribimos el siguiente código que nos permite crear un fichero binario a partir de una lista de datos.

```
11-creando-binario-lista.py > ...
1  # Creamos un fichero binario a partir de una lista
2
3  fichero = open("ficheros/lista.bin", "wb")
4  lista=[10,30,45,60,70,85,99]
5
6  # Convertimos la lista en un array binario
7  barray=bytearray(lista)
8
9  # escribimos el array binario en el fichero
10 fichero.write(barray)
11 fichero.close()
12
```

En la línea 7 se convierte la lista a un array binario, la función es `bytearray(lista)`.

En la línea 10 se escribe en el fichero el array binario.

El contenido del fichero generado es el siguiente:



Como se puede apreciar su contenido no se puede leer.

El siguiente ejemplo, leerá los datos del fichero creado antes y procederemos a escribir su contenido por pantalla.

El código se muestra a continuación:

```
11-leer-lista-en-binario-parte1.py > ...
1  # Leemos del fichero list.bin creando una nueva lista y mostramos por pantalla
2
3  fichero = open("ficheros/lista.bin", "rb")
4
5  # interpretado como no binario
6  nueva_lista = []
7  numero = list(fichero.read(1))
8
9  while numero:
10     nueva_lista += numero
11     numero = list(fichero.read(1))
12
13     # usando append()
14     # nueva_lista.append(numero)
15     # numero = list(fichero.read(1))
16
17  print(nueva_lista)
```

En la línea 7 se lee un número de la lista. En este caso el `read(1)` indica que hay que leer un número. El dato obtenido se pasa a una lista, como resultado, al leer la primera vez la variable `numero` tendrá el valor `[10]`.

En la línea 10 se añade el elemento al final de nueva_lista. Hay que hacerlo de esta forma, porque si ejecutamos el método `append()` lo que añade es `[numero]` y no el `numero`.

El resultado mostrado por pantalla es el siguiente

```
[10, 30, 45, 60, 70, 85, 99]  
rgh.informatica@macbpro-rgh Unidad06 %
```

Si utilizamos el método `append()` muestra esta salida

```
[[10], [30], [45], [60], [70], [85], [99]]  
rgh.informatica@macbpro-rgh Unidad06 %
```

Que es una lista donde cada elemento es una lista.

SERIALIZACIÓN DE OBJETOS PYTHON - pickle

Una librería muy utilizada a la hora de guardar objetos en Python es pickle. El enlace que lleva a la información oficial de la librería es <https://docs.python.org/3/library/pickle.html>.

Esta librería pertenece al conjunto de librerías del núcleo de Python y permite guardar la información de objetos Python directamente en ficheros binarios, y también cargarla de nuevo en un programa. La ventaja de poder compartir objetos Python como ficheros de sistema es que pueden ser transmitidos fácilmente y persistidos en disco para poder cargarlos cuando sean necesarios, por lo que es un recurso muy utilizado en la programación de Python.

Uno de los beneficios de usar pickle frente a JSON o XML es que los ficheros no son legibles por los humanos, por lo que es algo más seguro. Por otra parte, pickle, a diferencia de JSON, soporta cualquier tipo de dato, incluso tipos de datos propios. Uno de los puntos más importantes es que también soporta la serialización de funciones de Python, lo que hace que no tenga competidor viable en lo que respecta a archivos de texto.

Escritura de ficheros usando pickle

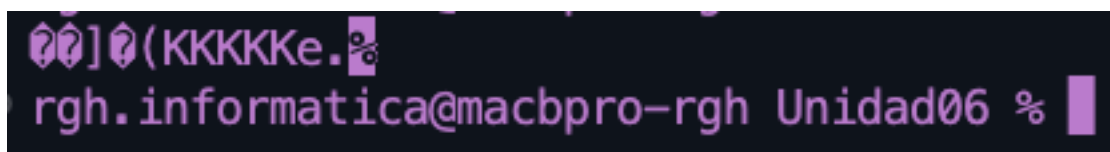
Veamos como funciona, para ello se necesita importar la librería pickle y vamos a escribir una lista de elementos.

El código para el uso de la librería pickle para escribir un fichero binario es el siguiente:

```
12-pickle-01-escribiendo-listas.py > ...
1  # escribiendo fichero binario ocn pickle
2
3  import pickle
4
5  # Podemos guardar lo que queramos, listas, diccionarios, tuplas...
6  lista = [1,2,3,4,5]
7
8  # Escritura en modo binario, vacía el fichero si existe
9  fichero = open('ficheros/lista.pckl', 'wb')
10
11 # Escribe la lista en el fichero
12 pickle.dump(lista, fichero)
13
14 # cerramos el fichero
15 fichero.close()
```

Como se puede apreciar en el código, se importa la librería `pickle`, se abre el fichero en modo escritura binaria y posteriormente se usa el método `dump()` para volcar la información en el fichero.

El resultado de la ejecución del programa anterior es lo que se muestra a continuación:



Notar que el fichero generado `lista.pckl` no tiene un formato legible.

Lectura de ficheros usando pickle

Ahora se trata de leer el contenido del fichero que se generó en el ejemplo anterior para reconstruir la estructura que se tenía inicialmente. El código para realizarlo es el siguiente:

```
12-pickle-02-leyendo-listas.py > ...  
1  # leyendo fichero formato pickle y transformado en lista  
2  import pickle  
3  
4  # Lectura en modo binario  
5  fichero = open ('ficheros/lista.pckl', 'rb')  
6  
7  # Cargamos los datos del fichero  
8  
9  lista = pickle.load(fichero)  
10 print (lista)  
11  
12 # cerramos el fichero  
13 fichero.close()
```

En la línea 5 se abre el fichero para una lectura binaria.

En la 9 se utiliza el método `load` para la lectura del fichero y almacenar los datos en la variable de nombre `lista`.

La ejecución del código anterior da el siguiente resultado.

```
12-pickle-02-leyendo-listas.py  
[1, 2, 3, 4, 5]  
rgh.informatica@macbpro-rgh Unidad06 %
```

Realicemos otro ejemplo trabajando ahora con un diccionario en vez de con una lista. El código es el siguiente:

```
12-pickle-03-escribiendo-dict.py > ...
1  # escribir y leer diccionario en fichero binario
2
3  import pickle
4
5  students = {
6      'Student 1': {
7          'Name': "Alice", 'Age' :10, 'Grade':4,
8      },
9
10     'Student 2': {
11         'Name': 'Bob', 'Age':11, 'Grade':5
12     },
13
14     'Student 3': {
15         'Name': 'Elena', 'Age':14, 'Grade':8
16     }
17 }
18
19 # serializa el diccionario
20
21 with open("ficheros/dict.pkl", "wb") as f:
22     pickle.dump(students, f)
23
24 # deserializa el diccionario y muestra por pantalla
25
26 with open("ficheros/dict.pkl", "rb") as f:
27     dict_deserializado = pickle.load(f)
28     print(dict_deserializado)
```

El resultado es:

```
endo-dict.py
{'Student 1': {'Name': 'Alice', 'Age': 10, 'Grade': 4}, 'Student 2': {'Name': 'Bob', 'Age': 11, 'Grade': 5}, 'Student
3': {'Name': 'Elena', 'Age': 14, 'Grade': 8}}
○ rgh.informatica@macbpro-rgh Unidad06 %
```

Persistencia de objetos

Para guardar objetos lo haremos dentro de una colección. La lógica sería la siguiente:

- 1. Crear una lista.*
- 2. Introducir los objetos en la lista.*
- 3. Guardar la lista haciendo un dump.*
- 4. Recuperar la lista haciendo un load.*
- 5. Seguir trabajando con nuestros objetos.*

Ejemplo catálogo de películas

A continuación, se muestra un ejemplo para ver como adaptar un catálogo de películas para usar persistencia utilizando pickle:

El código que implementa la secuencia anterior en el ejemplo de catálogo de películas es el siguiente:

La definición de clases usadas:

```
12-pickle-05-catalogo-peliculas.py > Catalogo > __init__
1  import pickle
2
3  class Pelicula:
4      def __init__(self, titulo, duracion, lanzamiento):
5          self.titulo = titulo
6          self.duracion = duracion
7          self.lanzamiento = lanzamiento
8          print('Se ha creado la película:', self.titulo)
9
10     def __str__(self):
11         return f"{self.titulo} {str(self.lanzamiento)}"
12
13 class Catalogo:
14     peliculas = []
15
16     def __init__(self):
17         self.cargar()
18
19     def agregar(self,p):
20         self.peliculas.append(p)
21         self.guardar()
22
23     def mostrar(self):
24         if len(self.peliculas) == 0:
25             print("El catálogo está vacío")
26         else:
27             for p in self.peliculas:
28                 print(p)
29
30     def cargar(self):
31         fichero = open('ficheros/catalogo.pckl', 'ab+')
32         fichero.seek(0)
33         try:
34             self.peliculas = pickle.load(fichero)
35         except:
36             print("El fichero está vacío")
37         finally:
38             fichero.close()
39             print(f"Se han cargado {str(len(self.peliculas))} películas")
40
41     def guardar(self):
42         fichero = open('ficheros/catalogo.pckl', 'wb')
43         pickle.dump(self.peliculas, fichero)
44         fichero.close()
45
```

El programa principal

```
12-pickle-05-catalogo-peliculas.py > Catalogo > __init__
46 def main():
47     # Creamos un catálogo
48     c = Catalogo()
49
50     # Mostramos el contenido
51     c.mostrar()
52
53     # Agregamos unas películas
54     c.agregar(Pelicula("El Padrino", 175, 1972))
55     c.agregar(Pelicula("El Padrino: Parte 2" , 202, 1974))
56
57     # Mostramos el catálogo de nuevo
58     c.mostrar()
59
60     # Borramos el catálogo de la memoria ram (persistir
61     del(c)
62
63     # Creamos un catálogo
64     c = Catalogo()
65
66     # Mostramos el contenido
67     c.mostrar()
68
69     # Agregamos una película
70     c.agregar(Pelicula("Prueba", 100, 2005))
71
72     # Mostramos el catálogo de nuevo
73     c.mostrar()
74
75 if __name__ == "__main__":
76     main()
77
```

En las líneas 31 y 32 se ha usado el modo **"ab+"**, que abre el fichero para escritura y lectura posicionándose al final. La línea 32 pone el cursor al comienzo del fichero.

Una opción mejor es abrir el fichero en modo **"rb"** lectura binaria

La salida que muestra el programa es la siguiente:

```
Se han cargado 3 películas
El Padrino 1972
El Padrino: Parte 2 1974
Prueba 2005
Se ha creado la película: El Padrino
Se ha creado la película: El Padrino: Part
El Padrino 1972
El Padrino: Parte 2 1974
Prueba 2005
El Padrino 1972
El Padrino: Parte 2 1974
Se han cargado 5 películas
El Padrino 1972
El Padrino: Parte 2 1974
Prueba 2005
El Padrino 1972
El Padrino: Parte 2 1974
Se ha creado la película: Prueba
El Padrino 1972
El Padrino: Parte 2 1974
Prueba 2005
El Padrino 1972
El Padrino: Parte 2 1974
Prueba 2005
○ rgh.informatica@macbpro-rg h Unidad06 %
```